CODETANTRA Home Learn Anywhere 202501090143@mitaoe.ac.in Support Logout

3.2.7. Student Data Analysis and Operations

04:03

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details**: Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students**: Determine the total number of students in the dataset.
- **Print all student roll numbers**: Extract and print the roll numbers of all students.
- **Print Subject 1 marks**: Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2**: Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3**: Identify the highest marks in Subject 3.
- **Print all subject marks**: Display the marks of all students for each subject.
- **Find total marks of students**: Compute the total marks for each student across all subjects.
- **Find the average marks of each student**: Compute the average marks for each student.
- **Find average marks of each subject**: Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2**: Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3**: Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3**: Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2**: Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2**: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1**: Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2**: Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject**: Count the number of students

Explorer

Operatio...

SUBMIT

Debugger

```python
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

#1. Print all student details
print("All student Details:\n",a)

#2. print total students

print("Total Students:", len(a))

#3. Print all student Roll numbers
print("All Student Roll Nos", a[:,0] )

#4. Print subject 1 marks
print("Subject 1 Marks", a[:,1])

#5. print minimum marks of Subject 2
print("Min marks in Subject 2", a[:,2].min()     )

#6. print maximum marks of Subject 3
print("Max marks in Subject 3", a[:,3].max()    )

#7. Print All subject marks
a1 = np.delete(a,0,1)
print("All subject marks:", a1     )

#8. print Total marks of students
print("Total Marks", a[:,1]+a[:,2]+a[:,3]      )
```

Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

02:22

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.
Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:
1. Searching: Find the indices where `search_value` appears in `array1` and print these indices.
2. Counting: Count how many times `count_value` appears in `array1` and print the count.
3. Broadcasting: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. Sorting: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Sample Test Cases +

Explorer

arrayOpe...

SUBMIT

```
    # Broadcasting
    # Takes a value to add to every single element in array1
    broadcast_value = int(input("Value to add: "))

    # 1. Find indices where value matches in array1
    # np.where returns a tuple, so we access index 0
    indices = np.where(array1 == search_value)[0]
    print(indices)

    # 2. Count occurrences in array1
    # Using np.count_nonzero on a boolean mask
    count = np.count_nonzero(array1 == count_value)
    print(count)

    # 3. Broadcasting addition
    # Adding the scalar broadcast_value to the entire array1
    broadcast_res = array1 + broadcast_value
    print(broadcast_res)

    # 4. Sort the first array
    # np.sort creates a new sorted version of the array
    sorted_array = np.sort(array1)
    print(sorted_array)

    # Count occurrences in array1

    # Broadcasting addition

    # Sort the first array
```

Terminal Test cases

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitw... 04:27

You are given two arrays `A` and `B`. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. **Arithmetic Operations**:
   - Compute the element-wise sum, difference, and product of the two arrays.
2. **Statistical Operations**:
   - Calculate the mean, median, and standard deviation of array `A`.
3. **Bitwise Operations**:
   - Perform bitwise `AND`, bitwise `OR`, and bitwise `XOR` on the arrays (ex: A_i `OR` B_i).

Input Format:

- The first line contains space-separated integers representing the elements of array `A`.
- The second line contains space-separated integers representing the elements of array `B`.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases +

Explorer

different...

SUBMIT

```python
import numpy as np

def array_operations(A, B):

    arr_a = np.array(A)

    arr_b = np.array(B)

    # Arithmetic Operations

    # Element-wise addition, subtraction, and multiplication
    sum_result = arr_a + arr_b
    diff_result = arr_a - arr_b
    prod_result = arr_a * arr_b

    # Statistical Operations

    # Calculate mean, median, and standard deviation for array A

    mean_A = np.mean(arr_a)
    median_A = np.median(arr_a)
    std_dev_A = np.std(arr_a)

    # Bitwise Operations

    # Perform bitwise AND, OR, and XOR on the two arrays
    and_result = np.bitwise_and(arr_a, arr_b)
    or_result = np.bitwise_or(arr_a, arr_b)
```

Terminal Test cases

3.2.3. Numpy: Custom Sequence Generation

02 05

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases +

customS...

SUBMIT

```
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
"""
# Generate the sequence using np.arange()
import numpy as np

# Take user input for the start, stop, and step of the sequence
# Each input is provided on a new line
try:
    start = int(input())
    stop = int(input())
    step = int(input())

    # Generate the sequence using np.arange()
    # Logic: np.arange(start, stop, step)
    # It starts at 'start' and goes up to, but does not include, 'stop'
    generated_sequence = np.arange(start, stop, step)

    # Print the generated sequence
    # NumPy arrays print in the [x y z] format shown in the test cases
    print(generated_sequence)

except ValueError:
    # Handle cases where input might not be a valid integer
    pass
# Print the generated sequence

```

Terminal Test cases

‹ PREV RESET SUBMIT NE

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

12:00

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking**: Stack the two matrices horizontally (side by side).
- **Vertical Stacking**: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases +

Explorer

stacking.py

SUBMIT

```
5   arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7   print("Enter Array2:")
8   arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9   """
10  import numpy as np
11
12  # --- Input matrices ---
13  # Prompt the user to input two 3x3 arrays row by row
14  print("Enter Array1:")
15  arr1 = np.array([list(map(int, input().split())) for i in range(3)])
16
17  print("Enter Array2:")
18  arr2 = np.array([list(map(int, input().split())) for i in range(3)])
19
20  # --- Perform horizontal stacking (hstack) ---
21  # Horizontal stacking combines arrays column-wise (side-by-side)
22  print("Horizontal Stack:")
23  h_stack = np.hstack((arr1, arr2))
24  print(h_stack)
25
26  # --- Perform vertical stacking (vstack) ---
27  # Vertical stacking combines arrays row-wise (one below the other)
28  print("Vertical Stack:")
29  v_stack = np.vstack((arr1, arr2))
30  print(v_stack)
31
32
33
```

Terminal Test cases

3.2.1. Numpy: Matrix Operations

07:40

The given code takes two 3×3 matrices, $\mathrm{matrix\_a}$, and $\mathrm{matrix\_b}$, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** ($\mathrm{matrix\_a} + \mathrm{matrix\_b}$)
2. **Subtraction** ($\mathrm{matrix\_a} - \mathrm{matrix\_b}$)
3. **Element-wise Multiplication** ($\mathrm{matrix\_a} * \mathrm{matrix\_b}$)
4. **Matrix Multiplication** ($\mathrm{matrix\_a} \cdot \mathrm{matrix\_b}$)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for $\mathrm{matrix\_a}$, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for $\mathrm{matrix\_b}$, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases +

Explorer

matrixOp...

SUBMIT

```
25  """
26  import numpy as np
27
28  # Input matrices
29  print("Enter Matrix A:")
30  matrix_a = np.array([list(map(int, input().split())) for _ in range(3)])
31
32  print("Enter Matrix B:")
33  matrix_b = np.array([list(map(int, input().split())) for _ in range(3)])
34
35  # Addition
36  print("Addition (A + B):")
37  print(matrix_a + matrix_b)
38
39  # Subtraction
40  print("Subtraction (A - B):")
41  print(matrix_a - matrix_b)
42
43  # Multiplication (element-wise)
44  print("Element-wise Multiplication (A * B):")
45  print(matrix_a * matrix_b)
46
47  # Matrix multiplication (dot product)
48  print("A dot B:")
49  print(np.dot(matrix_a, matrix_b))
50
51  # Transpose
52  print("Transpose of A:")
53  print(matrix_a.T)
```

Terminal Test cases

3.1.1. Numpy Array Operations

15:39

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases +

Explorer

numpyarr...

SUBMIT

```
1  import numpy as np
2
3
4  r,c=map(int,input().split())
5  elements=[]
6
7  for _ in range(r):
8
9      elements.extend(map(int,input().split()))
10
11 arr=np.array(elements).reshape(r,c)
12
13
14
15 print(arr)
16
17
18
19 print(arr.ndim)
20
21
22
23 print(arr.shape)
24
25
26 print(arr.size)
```

Terminal Test cases

PREV RESET SUBMIT NE